Lab Manual: Working with Delta Lake in Databricks

Objective

By the end of this lab, you will:

- Understand what Delta Lake is and why it's useful.
- Write and read Delta tables with PySpark.
- Handle schema evolution in Delta Lake.
- Use time travel to query past versions of your data.

Prerequisites

- A **Databricks workspace** with an active cluster.
- **Delta Lake runtime** (included in Databricks by default).
- Basic knowledge of Spark DataFrames.

Step 1: Initialise Spark Session

```
from pyspark.sql import SparkSession

# Create or retrieve a Spark session
spark = SparkSession.builder.appName("DeltaDemo").getOrCreate()
```

This initialises a Spark session named **DeltaDemo**.

Step 2: Create a Sample DataFrame

```
# Define sample employee data
data = [("Alice", 50000), ("Bob", 60000), ("Charlie", 70000)]

# Create DataFrame with columns name and salary
df = spark.createDataFrame(data, ["name", "salary"])

# Show DataFrame
```

```
df.show()
```

Creates a small employee dataset.

Step 3: Write Data to Delta Format

```
# Write DataFrame in Delta format
df.write.format("delta").mode("overwrite").save("dbfs:/mnt/delta/emp
loyees")
```

- `.format("delta")` → saves data as a Delta table.
- `.mode("overwrite")` → replaces existing data at the path.
- Stored at **dbfs:/mnt/delta/employees**.

Step 4: Read Data from Delta Table

```
# Read Delta table
df_read =
spark.read.format("delta").load("dbfs:/mnt/delta/employees")

# Show the data
df_read.show()
```

Reads the latest snapshot of the Delta table.

Step 5: Schema Evolution (Adding a New Column)

```
# New data with an additional 'department' column
data_new = [
    ("David", 80000, "HR"),
    ("Eve", 90000, "IT")
]

# Create new DataFrame with department column
df_new = spark.createDataFrame(data_new, ["name", "salary",
"department"])

# Append new data and allow schema evolution
```

```
df_new.write.format("delta") \
    .mode("append") \
    .option("mergeSchema", "true") \
    .save("dbfs:/mnt/delta/employees")
```

- `.option("mergeSchema", "true")` updates the table schema to include `department`.
- Older rows without `department` remain valid (null for missing values).

Step 6: Verify Schema Evolution

```
# Read updated Delta table
df_updated =
spark.read.format("delta").load("dbfs:/mnt/delta/employees")

# Show data with new schema
df_updated.show()
```

The table now has the `department` column alongside older rows.

Step 7: Time Travel (Querying Past Versions)

```
# Read Delta table as it was at version 0
df_version0 = spark.read.format("delta") \
    .option("versionAsOf", 0) \
    .load("dbfs:/mnt/delta/employees")

# Show version 0 data
df_version0.show()
```

- Delta's transaction log allows querying past versions.
- Useful for **rollbacks, audits, reproducibility**.

Verification Checklist

- Data written to Delta table.
- Data read back successfully.
- Schema evolution handled with `mergeSchema`.
- Time travel confirmed by reading `versionAsOf=0`.

Summary

In this lab, you:

- Created and stored a Delta table.
- Read data back using Spark.
- Handled schema evolution when new columns were added.
- Used time travel to query a previous version of the table.

With Delta Lake, your data lake gains:

- ACID transactions
- Schema enforcement
- Version control

This makes your data pipelines **reliable, auditable, and easy to manage**.